

บทที่ 4

การทดสอบและวิเคราะห์ผล

ในแต่ละขั้นตอนในการพัฒนา โครงการนั้น มีการทดสอบและวิเคราะห์ผลว่าสิ่งที่ได้ทำไปแล้วนั้นเป็นไปตามเป้าหมายของแต่ละขั้นหรือไม่ ซึ่งจำเป็น ต้องให้เป็นไปตามเป้าหมายนั้น แต่ถ้าไม่สามารถทำให้เป็นไป ได้ตามเป้าหมายได้ จะมีการพิจารณาในสิ่งที่เกิดขึ้นและหาทางแก้ไขใน โครงสร้างของระบบ หรือขั้นตอนและ เป้าหมายต่างๆ แต่ไม่ให้กระทบถึงเป้าหมายของโครงการ เมื่อได้ทำตามขั้นตอนต่างๆ เป็นที่เรียบร้อยแล้วจะมีการทดสอบและวิเคราะห์ผลอีกที ว่าระบบที่พัฒนาขึ้นมานั้นเป็นไปตามเป้าหมายของโครงการหรือไม่

4.1 การทดสอบการทำงานของฮาร์ดแวร์

ในการทดสอบการทำงานของฮาร์ดแวร์ทำได้โดยการตรวจสอบที่แอลอีดีของภาคไอทีเทอร์เนตคอนโทรเลอร์ ว่าติดหรือดับ ขณะที่ต่อแหล่งจ่ายไฟเข้ากับบอร์ดควบคุม จากนั้นทำการตรวจสอบการทำงานของไมโครคอนโทรเลอร์ หน่วยความจำแรมโดยทำการอ่านและเขียนข้อมูลจากโฮสต์ ไปยังบอร์ดควบคุม โดยโปรแกรม Armttool ทำการเปรียบเทียบข้อมูลที่เขียนลงไปและข้อมูลที่อ่านออกมา ทำให้สามารถตรวจสอบได้ว่าบอร์ดควบคุมนั้นยังทำงานได้อย่างถูกต้อง ดังคำสั่งข้างล่างสามารถที่จะนำไปเขียนเป็น script เพื่อตรวจสอบการทำงานของบอร์ดควบคุมได้ ดังนี้

```
#!/bin/sh
filename=$1
word=$2

# board initializatoin for ARM Control-board for Embedded System
# FLASH: 0x01000000      2MB   ROM   <----start
# RAM:  0x02000000      1MB   RAM   <----start

##### reset hardware #####

echo "initialize board"

./armtool.exe s

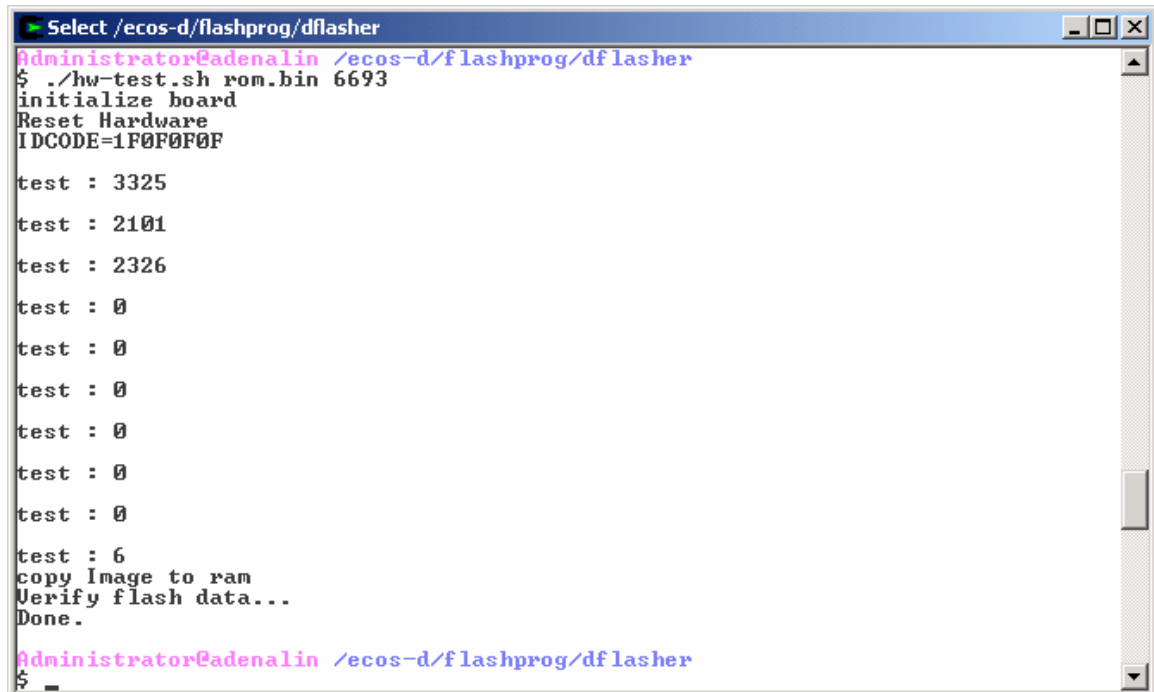
#####EBI initialize memory #####

./armtool.exe write 0xffe00000 0x01003325 #Flash initialize 0x1000000,16MB,2 Wait State
./armtool.exe write 0xffe00004 0x02002101 #Ram initialize 0x2000000,16MB,0 Wait State
./armtool.exe write 0xffe00008 0x03002326 #Ethernet initialize 0x3000000,16MB,2Wait State
```

```

./armtool.exe write 0xffe0000c 0x04000000 #not use
./armtool.exe write 0xffe00010 0x05000000 #not use
./armtool.exe write 0xffe00014 0x06000000 #not use
./armtool.exe write 0xffe00018 0x07000000 #not use
./armtool.exe write 0xffe0001c 0x08000000 #not use
./armtool.exe write 0xffe00020 1          ##Remap command
./armtool.exe write 0xffe00024 6          #Memory Controller
##### Write SRAM #####
echo "copy Image to ram"
./armtool.exe write 0x2000000 $word $filename
##### Read Sram #####
./armtool.exe read 0x02000000 $word $filename.save
echo "Verify flash data..."
cmp -c $filename $filename.save
echo "Done."

```



```

Select /ecos-d/flashprog/df flasher
Administrator@adenalin /ecos-d/flashprog/df flasher
$ ./hw-test.sh rom.bin 6693
initialize board
Reset Hardware
IDCODE=1F0F0F0F

test : 3325
test : 2101
test : 2326
test : 0
test : 0
test : 0
test : 0
test : 0
test : 0
test : 0
test : 6
copy Image to ram
Verify flash data...
Done.

Administrator@adenalin /ecos-d/flashprog/df flasher
$ _

```

รูปที่ 4.1 แสดงการตรวจสอบการทำงานของฮาร์ดแวร์

จากรูปที่ 4.1 แสดงให้เห็นการผลลัพธ์ที่ได้จากการรันสคริปต์ เพื่อทดสอบฮาร์ดแวร์ ในบรรทัดที่สามของ เอาท์พุตจากคำสั่ง เห็นได้ว่า IDCODE=1F0F0F0F ซึ่งเป็นชื่อของ ARM7TDMI ได้มาจากคำสั่ง ./armtool.exe s คือการรีเซตฮาร์ดแวร์ แสดงให้เห็นว่าการติดต่อบอร์ดควบคุมนั้นไม่มีปัญหาแต่อย่างใด ใน บรรทัดถัดมาของ script เป็นการตั้งค่าเริ่มต้นให้กับ EBI bus จนก่อนจะถึง คำสั่งทำการคัดลอกข้อมูลที่เก็บ อยู่ในไฟล์ที่ชื่อ rom.bin เข้าไปยัง ram เนื่องจาก AT91M40800 เป็นไมโครคอนโทรลเลอร์ที่จำเป็นต้องมีการตั้งค่าเริ่มต้นทุกครั้งหลังจากที่มีรีเซตหรือเปิดเครื่องขึ้นมาเพื่อให้สามารถอ้างแอดเดรสได้อย่างถูกต้อง เมื่อทำการอ่านค่าได้ทำการโปรแกรมเข้าไปแล้ว มาตรวจสอบหากไม่มีข้อผิดพลาดคือ ข้อมูลนั้นเหมือนกันแล้ว ก็ไม่มีข้อผิดพลาดอะไรขึ้นมาแสดงผลจากคำสั่ง cmd -C \$filename \$filename.save เมื่อถึงจุดนี้ก็สามารถที่จะบอกได้ว่า โปรแกรมเชอร์กับ หน่วยความจำนั้นไม่มีปัญหาแต่อย่างใด

จากนั้นจะเป็นการทดสอบการโปรแกรมหน่วยความจำแฟลช โดยจะทำการโปรแกรม ข้อมูลเข้าไปในแฟลช สามารถทำได้โดยรันคำสั่งดังต่อไปนี้

```
#!/bin/sh
filename=$1
word=$2

# board initializat on for ARM Control-board for Embedded System
# FLASH: 0x01000000      2MB   ROM   <----start
# RAM:   0x02000000      1MB   RAM   <----start

##### reset hardware #####

echo "initialize board"

./armtool.exe s

#####EBI initialize memory #####

./armtool.exe write 0xffe00000 0x01003325 #Flash initialize 0x1000000,16MB,2 Wait State
./armtool.exe write 0xffe00004 0x02002101 #Ram initialize 0x2000000,16MB,0 Wait State
./armtool.exe write 0xffe00008 0x03002326 #Ethernet initialize 0x3000000,16MB,2Wait State
./armtool.exe write 0xffe0000c 0x04000000 #not use
./armtool.exe write 0xffe00010 0x05000000 #not use
./armtool.exe write 0xffe00014 0x06000000 #not use
./armtool.exe write 0xffe00018 0x07000000 #not use
./armtool.exe write 0xffe0001c 0x08000000 #not use
./armtool.exe write 0xffe00020 1          ##Remap command
```

```

./armtool.exe write 0xffe00024 6          #Memory Controller
##### Write SRAM #####
echo "copy Image to ram"

./armtool.exe write 0x2000000 $word $filename
##### Write Number Word #####

./armtool.exe w 0x1ffc $word
#####Write Flaher #####

./armtool.exe w 0x80 9999 flasher.bin
##### Running Flaher #####

./armtool.exe x 0x80
#####Delay chip erase about 20 secound#####
echo "Waitting Chip erase...20s"

sleep 20s

##### Delay chip Programming depending on file ####
echo "Programming ....."

sleep 20s

##### Read Sram #####

./armtool.exe read 0x01000000 $word $filename.save

echo "Verify flash data..."

cmp -c $filename $filename.save

echo "Done."

##### E nd #####

```

จากสคริปต์นี้จะสังเกตได้ว่าแตกต่างจาก มีการเพิ่มคำสั่งให้โหลดแฟลชโปรแกรมเข้าไปที่แอดเดรส 0x80 ซึ่งจะเป็นส่วนของหน่วยความจำแอสแรม(SRAM)ที่อยู่ภายในไมโครคอนโทรลเลอร์ซึ่งหลังจากที่ได้ทำการรีแม็บ (Remap) หน่วยความจำแล้ว โดยแฟลชโปรแกรม(Flasher)จะทำหน้าที่ในการคัดลอกข้อมูลที่อยู่ในหน่วย ความจำแรม เข้าไปหน่วยความจำแฟลช จากการทดลองจำเป็นต้องเป็นเวลาอย่างน้อย 20 วินาที ขณะที่ทำการลบข้อมูลอยู่ในแฟลช และต้องอีกระยะเวลาอย่างน้อยเป็นเวลา 10 วินาทีสำหรับโปรแกรมข้อมูลขนาดประมาณ 300 กิโลไบต์ นั่นหมายความว่าถ้ามีข้อมูลขนาด 1 เมกกะไบต์ สคริปต์ที่เขียนขึ้นก็ต้องทำการแก้ไขส่วนของหน่วยเวลาในการโปรแกรม (Delay) อย่างน้อย ประมาณ 30 วินาที จึงจะทำการ โปรแกรมแฟลช ได้เสร็จ ดังรูปข้างล่างแสดงการ โปรแกรมแฟลช ขนาด 26,772 ไบต์

```

Select /ecos-d/flashprog/df flasher
Administrator@adenalin /ecos-d/flashprog/df flasher
$ ../flasher-test.sh rom.bin 6693
initialize board
Reset Hardware
IDCODE=1F0F0F0F

test : 3325
test : 2101
test : 2326
test : 0
test : 0
test : 0
test : 0
test : 0
test : 0
test : 6
copy Image to ram
test : 1a25
end of file at 0x37c
Run program at 0x80
Waiting Chip erase...20s
Programming .....
Verify flash data...
Done.
Administrator@adenalin /ecos-d/flashprog/df flasher
$

```

รูปที่ 4.2 แสดงการตรวจสอบการทำงานโปรแกรมแฟลช

หากไม่มีการรายงานข้อผิดพลาดขึ้นมาในขณะที่ทำการตรวจสอบข้อมูลที่ได้ทำการโปรแกรม แสดงว่าทุกส่วนของบอร์ดควบคุมนั้น สามารถทำงานได้ไม่มีปัญหาแต่อย่างใด เพราะว่าสามารถทำการโปรแกรมแฟลชได้โดยอาศัยการทำงานของ ไมโครคอนโทรเลอร์ และหน่วยความจำที่อยู่ภายในบอร์ดควบคุมเป็นตัวทำงาน

ซึ่งเพียงเท่านี้ก็สามารที่จะเขียนโปรแกรมเข้าไปควบคุมการทำงานของบอร์ดควบคุมได้แล้ว แต่การทำงานที่ได้ทดลองไปแล้วนั้น ยังคงต้องทำงานที่หน่วยความจำแรมเท่านั้น หมายถึงว่า หากถอดแหล่งจ่ายหรือหรือถอดปรั้เซตแล้ว โปรแกรมก็ไม่สามารถทำงานได้

4.2 การทดสอบการทำงานของซอฟต์แวร์

การเขียนโปรแกรมเข้าไปทำงานที่หน่วยความจำแรมโดยอาศัย Armtool ที่ได้พัฒนาขึ้น จะเหมาะสมสำหรับงานในการพัฒนาโปรแกรม ไม่เหมาะสมที่ต้องไปโปรแกรมแฟลชทุกครั้ง เพราะการอ่านหรือเขียนหน่วยความจำแรม ไม่มีข้อจำกัดเหมือนการเขียนแฟลช แม้ว่าจะสามารถโปรแกรมได้ใหม่เป็น 100,000 ครั้งก็ตาม จำเป็นต้องคำนึงการใช้งานของแอปพลิเคชัน ที่บางครั้งจำเป็นต้องเขียนข้อมูลในแฟลช เพื่อเก็บสถานะของงานเอาไว้ ดังนั้นการพัฒนาโปรแกรมจำเป็นต้องทดสอบการทำงานของโปรแกรมที่เขียนขึ้นโดยให้

ทำงานที่แรม เมื่อโปรแกรมทำงานถูกต้องและพร้อมที่จะให้ทำงานได้ด้วยตัวเอง จึงจะทำการโปรแกรมเข้าไปที่หน่วยความจำแฟลช

การพัฒนาโปรแกรมให้บอร์ดควบคุมสามารถทำงานได้ด้วยตัวเอง (Stand Alone) นั้น สามารถทำได้ดังนี้

1. ต้องเขียนบูตโค้ด (Boot Code) ขึ้นมาเพื่อทำการกำหนดค่าเริ่มต้นให้กับฮาร์ดแวร์ ทดสอบและตรวจสอบข้อผิดพลาดของฮาร์ดแวร์ จากนั้นให้กระโดดไปทำงานที่แอปพลิเคชันที่เขียนขึ้น ปกติแล้วบูตโค้ดจะเป็นภาษาแอสเซมบลี หน้าที่หลักสามารถแบ่งออกได้ดังนี้

- ทำการรีเซ็ตหน่วยความจำ โดยคำสั่งแรกของเมื่อเปิดบอร์ดควบคุมให้ทำงานจะอยู่ที่แอดเดรส 0x00 ซึ่งเป็นรีเซตแอดเดรสของโมโครโปรเซสเซอร์ ARM7TDMI โดยก่อนคำสั่งรีเซ็ตแอดเดรส 0x00 จะถูกแมปไปยังหน่วยความจำแฟลชขนาด 1 เมกกะไบต์ ซึ่งหลังจากทำการรีเซ็ตแล้ว หน่วยความจำแฟลชจะถูกแมปไปที่แอดเดรส 0x01000000

- ทำการตรวจสอบอุปกรณ์ที่อยู่บนบอร์ด เพื่อรายงานข้อผิดพลาดที่เกิดขึ้นกับตัวฮาร์ดแวร์ จากนั้นก็ทำการกระโดดไปทำงานที่แอปพลิเคชันโปรแกรม ที่เขียนขึ้น

2. แอปพลิเคชันโปรแกรม เป็นส่วนของโปรแกรมที่เขียนขึ้นให้บอร์ดควบคุมนั้นทำงานได้ตามความต้องการการ

โปรแกรมการใช้งานให้บอร์ดควบคุมสามารถทำงานได้ด้วยตัวเอง(Stand Alone)

/* ชื่อไฟล์ boot.s เป็นส่วนที่ทำหน้าที่ในการรีเซ็ตหน่วยความจำ */

```
.equ      EBI_BASE,0xFFE00000      /* EBI แอดเดรส */

.global  __main

__main:   .long   InitReset          /* reset vector */
undefvec: .long   undefvec           /* Undef vector*/
swivec:   .long   swivec             /* SW interrupt vector*/
pabtvec:  .long   pabtvec             /* Prefetch abort vector */
dabtvec:  .long   dabtvec            /* Data abort vector*/
rsvdvec:  .long   rsvdvec            /* reserved */
irqvec:   ldr     pc, [pc,#-0xF20]    /* IRQ : read the AIC */
fiqvec:   ldr     pc, [pc,#-0xF20]    /* FIQ : read the AIC */
InitReset: mov     sp,#0x400          /* Initialise Stack ที่ 0x020ffffc ถึง ram*/
          ldr     r10, PInitTableEBI /* get the address of the chip select register image */
          movs    r0, pc, LSR #20     /* pc > 0x100000 */
          moveq   r10, r10, LSL #12   /* Mask the 12 highest bits of the address */
```

```

        moveq    r10, r10, LSR #12

        ldr      r12, PtInitRemap      /* get the real jump address ( after remap ) */

        ldmia    r10!, {r0-r9,r11}    /* load the complete image and the EBI base */

        stmia    r11!, {r0-r9}        /* store the complete image with the remap command */

        mov      pc, r12               /* jump and break the pipeline */

InitRemap: b      main

PtInitTableEBI: .long InitTableEBI    /* Table for EBI initialization */

PtInitRemap:    .long InitRemap        /* address where to jump after REMAP */

InitTableEBI:   .long 0x01003325       /*0x01000000, 16MB, 0 hold, 16 bits, 2 WS */
                .long 0x02002121       /*0x02000000, 16MB, 0 hold, 16 bits, 0 WS */
                .long 0x03002326       /*0x03000000, 16MB, 0 hold, 16 bits, 2 WS */
                .long 0x40000000        /* unused */
                .long 0x50000000        /* unused */
                .long 0x60000000        /* unused */
                .long 0x70000000        /* unused */
                .long 0x80000000        /* unused */
                .long 0x00000001        /* REMAP command */
                .long 0x00000006        /* 7 memory regions, standard read */
                .long EBI_BASE         /* EBI Base address */

/* สิ้นสุดไฟล์ boot.s */

```

ส่วนของไฟล์สำหรับแอปพลิเคชันโปรแกรมสามารถเขียนเป็นภาษาซีได้ดังนี้

```

/* ไฟล์แอปพลิเคชันโปรแกรม application.c */

int main()
{
    /* โค้ดสำหรับ แอปพลิเคชันโปรแกรม */

    .....

    return 0 ;

}

/* สิ้นสุดไฟล์ application.c */

```

ส่วนของไฟล์ Makefile ที่จะใช้ในการคอมไพล์โค้ดโปรแกรม

```
/* ชื่อไฟล์ Makefile สำหรับการคอมไพล์โค้ดอัตโนมัติ */

All:    main.bin

Main.bin : boot.s main.c

        arm-elf-gcc -O2 -c -mcpu=arm7tdmi boot.s main.c
        arm-elf-ld -Ttext 0x01000000 -o main.elf boot.o main.o
        arm-elf-objcopy -O binary main.elf $@

/* สิ้นสุด Makefile */
```

เมื่อถึงขั้นตอนนี้แล้วจะได้เอาท์พุทไฟล์ ชื่อ main.bin จากนั้น ใช้แฟลชโปรแกรมมิง โปรแกรมข้อมูลเข้าไปยังแฟลช ทดสอบโปรแกรมโดยการรีเซตฮาร์ดแวร์ ดูว่าสามารถทำงานได้ถูกต้องตามต้องการได้หรือไม่ เป็นอันสิ้นสุดกระบวนการการสร้าง Stand Alone แอปพลิเคชัน